

Министерство науки и высшего образования Российской Федерации
Пензенский государственный
университет Кафедра «Вычислительная техника»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К курсовому проектированию
По курсу «Логика и основы алгоритмизации
в инженерных задачах»
на тему «Реализация алгоритма раскрашивания графа»

23.12.25
отлично
В.И.А.

Выполнил:
студент группы 24ВВВ3
Виноградов Б.С.
Приняла:
к.т.н. доцент Юрова О.В.

Пенза 2025

ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
Факультет Вычислительной техники
Кафедра "Вычислительная техника"

"УТВЕРЖДАЮ"

Зав. кафедрой ВТ

« » 20

ЗАДАНИЕ

на курсовое проектирование по курсу

"Поиск и основы оптимизации в инженерных задачах"
Студенту Виноградову Богдану Сергеевичу Группа 24ВВВЗ
Тема проекта Реализация алгоритма раскрывающих графов

Исходные данные (технические требования) на проектирование

Разработка алгоритмов и программного обеспечения
в соответствии с данными заданием курсового
проекта.

Пояснительная записка должна содержать:

1. Постановку задачи;
2. Теоретическую часть задания;
3. Описание алгоритма поставленной задачи;
4. Пример ручного расчета задачи и вычислений (на
небольшом участке работы алгоритма);
5. Описание самой программы;
6. Тесты;
7. Список литературы;
8. Листинг программы;
9. Результаты работы программы.

Объем работы по курсу

1. Расчетная часть

Ручной расчет работы алгоритма

2. Графическая часть

Схема алгоритма в формате блок-схем.

3. Экспериментальная часть

Тестирование программы:

Результаты работы программы на тестовых данных

Срок выполнения проекта по разделам

- 1 Исследование теоретической части курсового к
- 2 Разработка алгоритмов программы к
- 3 Разработка программы к
- 4 Тестирование и завершение работы программы к
- 5 Оформление пояснительной записки к
- 6
- 7
- 8

Дата выдачи задания "12" сентября 2025

Дата защиты проекта " "

Руководитель

Ильина О.В.

Задание получил

"12" сентября

2025 г.

Студент

Виноградов Богдан Сергеевич

Вич

Содержание

Реферат.....	5
Введение.....	6
1. Постановка задачи.....	8
2. Теоретическая часть задания.....	10
3. Описание алгоритма программы.....	12
4. Описание программы.....	19
5. Тестирование.....	24
6. Ручной расчёт задачи.....	30
Заключение.....	34
Список используемых источников.....	36
Приложение А. Листинг программы.....	37

Реферат

Отчёт 48 страниц, 13 рисунков, 2 таблицы, 5 источников.

ГРАФ, ТЕОРИЯ ГРАФОВ, НЕЗАВИСИМОЕ МНОЖЕСТВО,
ХРОМАТИЧЕСКОЕ ЧИСЛО, АЛГОРИТМ РАСКРАСКИ.

Цель исследования — разработка программы для раскрашивания графа с использованием алгоритма поиска наибольшего независимого множества, позволяющего определить хроматическое число графа.

В работе рассмотрены основы теории графов, понятие независимого множества и хроматического числа графа. Реализован алгоритм поиска наибольшего независимого множества методом полного перебора, который используется для пошагового раскрашивания вершин графа. Программа позволяет создавать граф автоматически или загружать его из файла, нормализовать его матрицу смежности, находить наибольшее независимое множество, раскрашивать граф и сохранять результаты.

Установлено, что использование наибольшего независимого множества при раскраске графа обеспечивает корректное определение хроматического числа и эффективное распределение цветов, что делает данный подход подходящим для учебных и исследовательских целей.

Введение

Раскрашивание графа является одной из фундаментальных задач теории графов, имеющей как теоретическое значение, так и практические приложения в различных областях — от оптимизации расписаний и распределения ресурсов до анализа сетевых структур и визуализации данных. Хроматическое число графа определяется как минимальное количество цветов, необходимых для раскрашивания всех вершин графа так, чтобы смежные вершины имели разные цвета. Задача корректного раскрашивания графа является NP-трудной, что делает разработку эффективных алгоритмов особенно актуальной для небольших и средних графов.

В рамках данной курсовой работы реализован алгоритм раскрашивания графа, основанный на пошаговом поиске наибольшего независимого множества, который позволяет определить хроматическое число графа. Программа позволяет создавать граф автоматически или загружать его из файла, нормализовать матрицу смежности, находить наибольшее независимое множество и раскрашивать граф с оптимальным распределением цветов.

В качестве среды разработки выбран Visual Studio Code, а язык программирования — C++. Этот выбор обусловлен широкими возможностями языка C++ для работы со структурами данных и эффективностью выполнения операций с графами. Программа обеспечивает консольный вывод матриц смежности, раскрашенных графов и хроматического числа, а также позволяет сохранять результаты в файл.

Целью работы является разработка программы, которая:

1. генерирует случайные графы;
2. загружает графы из файлов;
3. нормализует матрицу смежности графа;
4. находит наибольшее независимое множество для пошагового раскрашивания;
5. раскрашивает граф и определяет его хроматическое число;

6. обеспечивает сохранение результатов в файл.

Программа будет полезным инструментом для изучения свойств графов, демонстрации работы алгоритма раскрашивания и решения практических задач, связанных с оптимальным распределением ресурсов и анализом сетевых структур.

1. Постановка задачи

Граф в программе задаётся множеством вершин, между которыми могут существовать рёбра (для неориентированного графа) или дуги (для ориентированного графа). Состояние графа определяется его матрицей смежности, где каждая запись указывает наличие или отсутствие связи между двумя вершинами. Значение «1» в элементе матрицы означает наличие ребра или дуги, значение «0» — её отсутствие. Для неориентированного графа матрица смежности должна быть симметричной относительно главной диагонали, а диагональные элементы равны нулю, что соответствует отсутствию петель. Для ориентированного графа симметричность матрицы не требуется.

Граф может быть сформирован одним из двух способов:

1. автоматически, с использованием случайной или псевдослучайной генерации матрицы смежности;
2. путём загрузки заранее подготовленного графа из текстового файла.

При случайной генерации каждое возможное ребро или дуга создаётся случайным образом. При псевдослучайной генерации вероятность появления ребра или дуги задаётся пользователем в диапазоне от 1 до 10, что позволяет контролировать плотность графа. Для неориентированного графа при генерации обеспечивается симметричность матрицы смежности. После создания или загрузки графа программа выводит сформированную матрицу смежности на экран.

После получения графа программа выполняет анализ его структуры с целью определения возможности корректного раскрашивания и вычисления хроматического числа. Алгоритм раскрашивания основан на пошаговом поиске наибольшего независимого множества, который используется для последовательного присвоения цветов вершинам графа.

Программа должна обеспечивать следующие функциональные возможности:

1. выбор типа графа (ориентированный или неориентированный);
2. выбор способа создания графа (автоматическая генерация или загрузка из файла);
3. сохранение графа в файл в виде матрицы смежности;
4. загрузка графа из файла;
5. вывод матрицы смежности на экран;
6. нахождение наибольшего независимого множества для пошагового раскрашивания;
7. раскраска графа с присвоением цветов вершинам и вычислением хроматического числа;
8. повторный анализ различных графов без завершения работы программы.

Управление программой осуществляется с помощью клавиатуры через текстовое меню. Интерфейс обеспечивает удобный доступ ко всем функциям, позволяя пользователю создавать, анализировать, раскрашивать и сохранять графы без необходимости завершения работы программы.

2. Теоретическая часть задания

Граф представляет собой фундаментальную математическую структуру, состоящую из множества вершин и множества рёбер, соединяющих эти вершины. В данной работе рассматриваются как неориентированные, так и ориентированные графы. В неориентированных графах рёбра не имеют направления и обозначают двустороннюю связь между вершинами, тогда как в ориентированных графах каждое ребро имеет заданное направление, определяющее переход от одной вершины к другой.

Для алгоритмической обработки графов используется представление в виде матрицы смежности. Матрица смежности — это квадратная матрица размерности $N \times N$, где N — количество вершин графа. Элемент матрицы $M[i][j]$ равен 1, если существует ребро из вершины i в вершину j , и 0 в противном случае. Для неориентированных графов матрица симметрична относительно главной диагонали, а диагональные элементы равны нулю, что соответствует отсутствию петель.

Центральным понятием работы является наибольшее независимое множество (*MIS*) графа — множество вершин, в котором ни одна пара вершин не соединена ребром, и которое не может быть расширено добавлением другой вершины. Использование *MIS* позволяет последовательно раскрашивать граф: на каждой итерации выбирается *MIS* из оставшихся вершин, и всем вершинам этого множества присваивается один цвет. Повторение этого процесса для оставшихся вершин приводит к корректной раскраске графа и вычислению хроматического числа.

Алгоритм работы программы включает следующие шаги:

1. создание графа:
 - автоматически, с помощью случайной генерации матрицы смежности;
 - или загрузка графа из файла.

2. Нормализация матрицы смежности: преобразование всех ненулевых элементов в 1, чтобы корректно определить смежность вершин.

3. Поиск наибольшего независимого множества (MIS) методом полного перебора.

4. Раскраска графа:

- на каждой итерации выбирается MIS из оставшихся вершин;
- всем вершинам множества присваивается один цвет;
- вершины удаляются из множества оставшихся, и процесс повторяется до полного раскрашивания.

5. Вычисление хроматического числа — максимального числа использованных цветов.

Программа позволяет работать с графами любого типа, обеспечивает сохранение матрицы и результатов раскраски в файл. Такой подход наглядно демонстрирует связь между теоретическими понятиями *MIS* и практической задачей раскраски графа, а также обеспечивает корректное и эффективное определение хроматического числа.

3. Описание алгоритма программы

Программа реализует алгоритм раскрашивания графа, основанный на поиске наибольшего независимого множества (MIS). В качестве основной структуры данных используется матрица смежности — двумерный динамический массив `int**`, где элемент `G[i][j]` равен 1, если существует ребро между вершинами `i` и `j`, и 0 в противном случае. Для неориентированных графов матрица симметрична, а диагональные элементы равны нулю.

3.1. Создание графа

Пользователю предлагается выбрать способ создания графа:

1. автоматическая генерация графа:
 - программа случайным образом создаёт рёбра между вершинами;
 - для ориентированного графа матрица может быть асимметричной;
 - для неориентированного графа обеспечивается симметрия матрицы смежности;
 - диагональные элементы устанавливаются равными нулю, чтобы исключить петли.
2. Загрузка графа из файла:
 - файл должен содержать количество вершин и матрицу смежности;
 - программа проверяет корректность данных (положительное количество вершин, только целые числа ≥ 0);
 - при ошибках ввода программа выводит сообщения и предлагает повторить загрузку.

После создания или загрузки графа программа выводит исходную матрицу смежности на экран и выполняет её нормализацию, преобразуя все ненулевые элементы в 1, чтобы корректно определить смежность вершин.

3.2. Алгоритм раскраски графа

Алгоритм раскраски выполняется следующим образом:

1. создаётся массив оставшихся вершин, ещё не раскрашенных.
2. На каждой итерации выполняется поиск MIS среди оставшихся вершин методом полного перебора:
 - для каждой возможной комбинации вершин проверяется, является ли множество независимым;
 - выбирается множество максимального размера.
3. Всем вершинам найденного MIS присваивается один новый цвет.
4. Вершины MIS удаляются из множества оставшихся вершин, и процесс повторяется до полного раскрашивания графа.

3.3. Вычисление хроматического числа

Хроматическое число определяется как максимальный номер использованного цвета после завершения раскраски. Программа также сохраняет и выводит:

1. список вершин в MIS;
2. раскраску всех вершин по цветам;
3. размер наибольшего независимого множества.

3.4. Функциональные возможности программы

Программа обеспечивает:

1. выбор типа графа (ориентированный или неориентированный);
2. выбор способа создания графа (автоматически или из файла);
3. нормализацию матрицы смежности;
4. поиск наибольшего независимого множества;
5. пошаговую раскраску графа с учётом MIS;

6. определение хроматического числа;
7. вывод результатов на экран;
8. сохранение матрицы и результатов раскраски в файл.

3.5. Особенности реализации

1. Используется динамическое выделение памяти для матрицы и массивов вершин;
2. Программа проверяет корректность ввода данных и защищена от ошибок;
3. Программа корректно обрабатывает графы без рёбер — в этом случае хроматическое число равно 1, а MIS содержит все вершины;
4. Программа позволяет многократно анализировать различные графы без перезапуска.

Таким образом, программа реализует полный цикл работы с графами, начиная с их генерации или загрузки, нормализации, поиска MIS и пошаговой раскраски, и заканчивая выводом результатов и их сохранением. Она демонстрирует практическое применение понятий наибольшего независимого множества и хроматического числа для задач раскраски графа.

Функция *isIndependent*

ВХОД: G — матрица смежности графа, $vertexSet$ — массив целых чисел, содержащий номера вершин множества, $setSize$ — количество вершин в массиве $vertexSet$.

ВЫХОД: логическое значение (ИСТИНА, если множество независимое).

1. Проверяем все пары вершин в множестве на наличие ребер между ними
2. ДЛЯ i от 0 до $setSize-1$ ВЫПОЛНЯТЬ:
3. Текущая вершина из множества: $vertexSet[i]$
4. ДЛЯ j от $i+1$ до $setSize-1$ ВЫПОЛНЯТЬ:
5. Следующая вершина из множества: $vertexSet[j]$

6. Получаем значение из матрицы смежности для пары вершин
7. $edgeValue = G[vertexSet[i]][vertexSet[j]]$
8. Если существует ребро между вершинами, множество не является независимым
9. ЕСЛИ $edgeValue \neq 0$ ТОГДА:
 10. Найдено ребро, множество зависимо
 11. ВЕРНУТЬ ЛОЖЬ
 12. КОНЕЦ УСЛОВИЯ
13. Если граф неориентированный и симметричный, дополнительная проверка не требуется
14. В ориентированном графе проверяем только прямое ребро
15. КОНЕЦ ВНУТРЕННЕГО ЦИКЛА
16. КОНЕЦ ВНЕШНЕГО ЦИКЛА
17. Если ни для одной пары вершин не найдено ребра, множество независимое
18. ВЕРНУТЬ ИСТИНА

Функция *findMIS*

ВХОД: G — матрица смежности; $numG$ — количество вершин в графе; $currentSet$ — текущее независимое множество; $currentSize$ — текущее количество вершин в множестве; $remaining$ — массив оставшихся вершин для рассмотрения; $remainingSize$ — количество вершин в массиве $remaining$; $maxSet$ — найденное наибольшее независимое множество; $maxSize$ — размер найденного множества.

ВЫХОД: нет (результат возвращается через параметры $maxSet$ и $maxSize$).

1. Базовое условие обновления максимального множества
2. Если текущее множество больше сохраненного максимального, обновляем
3. ЕСЛИ $currentSize > maxSize$ ТОГДА:

4. Запоминаем новый максимальный размер
5. $maxSize = currentSize$
6. Копируем все вершины текущего множества в $maxSet$
7. ДЛЯ i от 0 до $currentSize-1$ ВЫПОЛНЯТЬ:
8. $maxSet[i] = currentSet[i]$
9. КОНЕЦ ЦИКЛА
10. КОНЕЦ УСЛОВИЯ
11. Рекурсивный перебор оставшихся вершин
12. ДЛЯ i от 0 до $remainingSize-1$ ВЫПОЛНЯТЬ:
13. Добавляем текущую вершину в множество кандидатов
14. $currentSet[currentSize] = remaining[i]$
15. Создаем новый массив для вершин, которые еще можно добавить
16. Это вершины, не смежные с только что добавленной вершиной $remaining[i]$
17. СОЗДАТЬ массив $newRemaining$ размером $remainingSize$
18. $newSize = 0$
19. Проходим по всем вершинам после текущей, чтобы избежать дублирования
20. ДЛЯ j от $i+1$ до $remainingSize-1$ ВЫПОЛНЯТЬ:
21. Проверяем, есть ли ребро между $remaining[i]$ и $remaining[j]$
22. ЕСЛИ $G[remaining[i]][remaining[j]] = 0$ ТОГДА:
23. Если ребра нет, вершину можно добавить в следующий шаг
24. $newRemaining[newSize] = remaining[j]$
25. $newSize += 1$
26. КОНЕЦ УСЛОВИЯ
27. КОНЕЦ ЦИКЛА
28. Рекурсивный вызов для расширения текущего множества
29. Теперь рассматриваем вершины из $newRemaining$
30. ВЫЗВАТЬ $findMIS()$
32. Освобождаем временный массив

33. УДАЛИТЬ массив *newRemaining*

34. КОНЕЦ ЦИКЛА

Функция *printMIS*

ВХОД: *maxSet* — наибольшее независимое множество; *maxSize* — его размер.

ВЫХОД: нет (информация выводится на экран).

1. ВЫВЕСТИ "Наибольшее независимое множество:"

2. ВЫВЕСТИ "Размер: " + значение *maxSize*

3. ЕСЛИ *maxSize* = 0:

4. ВЫВЕСТИ "Множество пустое"

5. ВЕРНУТЬ

6. КОНЕЦ УСЛОВИЯ

7. ВЫВЕСТИ "Вершины: "

8. ДЛЯ *i* от 0 до *maxSize*-1:

9. ВЫВЕСТИ *maxSet[i]*

10. ЕСЛИ *i* < *maxSize*-1: ВЫВЕСТИ ", "

11. КОНЕЦ ЦИКЛА

12. ПЕРЕЙТИ на новую строку

Функция *colorGraph*

ВХОД: *G* — матрица смежности графа, *numG* — количество вершин.

ВЫХОД: *colors* — массив цветов для каждой вершины.

1. СОЗДАТЬ массив *colors* размером *numG*, заполнить нулями

2. СОЗДАТЬ массив *remaining* со всеми вершинами (от 0 до *numG*-1)

3. *remainingSize* = *numG*

4. *currentColor* = 1

5. ПОКА *remainingSize* > 0:

6. НАЙТИ наибольшее независимое множество среди *remaining*

7. ПРИСВОИТЬ *currentColor* всем вершинам найденного множества

8. УДАЛИТЬ раскрашенные вершины из *remaining*

9. *currentColor* += 1
10. КОНЕЦ ЦИКЛА
11. ОСВОБОДИТЬ память *remaining*
12. ВЕРНУТЬ *colors*

Функция *processColoring*

ВХОД: numG — количество вершин; G — матрица смежности;
outColors, outChromaticNumber, outMisSet, outMisSize — выходные
параметры.

ВЫХОД: нет (результаты возвращаются через параметры).

1. ЕСЛИ граф пуст:
2. УСТАНОВИТЬ все выходные параметры в нулевые значения
3. ВЫВЕСТИ ошибку
4. ВЕРНУТЬ
5. КОНЕЦ УСЛОВИЯ
6. НАЙТИ MIS в графе
7. СОХРАНИТЬ MIS в outMisSet и outMisSize
8. ВЫЗВАТЬ printMIS для вывода информации
9. colors = ВЫЗВАТЬ colorGraph для раскраски графа
10. ВЫЧИСЛИТЬ хроматическое число
11. ВЫВЕСТИ раскраску всех вершин
12. ВЫВЕСТИ хроматическое число
13. Сохранить результаты
14. outColors = colors
15. outChromaticNumber = хроматическое число

4. Описание программы

Для написания данной программы использован язык программирования C++. Язык C++ предоставляет широкие возможности для работы с динамическими структурами данных и реализации рекурсивных алгоритмов, что критически важно для задач перебора в теории графов. Программа разработана как консольное приложение в среде разработки *Visual Studio Code*, что обеспечивает простоту отладки и кроссплатформенность.

Программа является многомодульной, поскольку содержит несколько функций, каждая из которых выполняет отдельную задачу:

- *main()* — основное меню и управление программой
- *autoGraphGeneration()* — автоматическая генерация графа
- *fileGraphGeneration()* — загрузка графа из файла
- *saveResultsToFile()* и *saveMatrixToFile()* — сохранение результатов
- *printMatrix()* — вывод матрицы смежности
- *normalizeGraph()* — нормализация матрицы смежности
- *isIndependent()* — проверка независимости множества вершин
- *findMIS()* — поиск наибольшего независимого множества
- *printMIS()* — вывод информации о *MIS*
- *colorGraph()* — алгоритм раскраски графа
- *processColoring()* — основная функция обработки раскраски
- *clearMatrix()* — освобождение памяти

4.1. Основная логика работы программы

После запуска программы пользователю предлагается главное меню:

Как создать граф?

1 — Автоматически

2 — Из файла

3 — Выход из программы

Если пользователь выбирает создание графа, программа выводит запрос на количество вершин в графе и предлагает дополнительные параметры:

```
cout << "Введите количество вершин графа: ";
cin >> numG;
```

Далее программа запрашивает:

Тип графа:

1 — Ориентированный

2 — Неориентированный

и

Взвешенность графа:

1 — Взвешенный

2 — Невзвешенный

4.2. Генерация матрицы смежности

При автоматической генерации для каждой пары вершин i и j создаётся случайное значение:

```
if (isWeighted) {
    int edgeProb = rand() % 2;
    if (edgeProb != 0) {
        G[i][j] = rand() % 10 + 1; // случайный вес от
1 до 10
    }
} else {
    G[i][j] = rand() % 2; // 0 или 1
}
```

Для неориентированного графа обеспечивается симметричность матрицы:

```
if (!isOriented) {
    G[j][i] = G[i][j];
}
```

4.3. Загрузка графа из файла

Если пользователь выбирает загрузить граф из файла, программа считывает файл, содержащий число вершин и матрицу смежности.

Пример обработки файла:

```

        ifstream fin(filename);
        if (!fin.is_open()) {
            cout << "Не удалось открыть файл. Попробуйте
снова\n";
            continue;
        }
        fin >> fileVertexCount; // чтение количества вершин

```

После загрузки графа программа выводит исходную матрицу смежности и выполняет её нормализацию — преобразование всех ненулевых значений в 1 для корректной работы алгоритмов поиска независимого множества.

4.4. Алгоритм поиска наибольшего независимого множества

Перед выполнением раскраски программа находит наибольшее независимое множество (*MIS*) с помощью рекурсивного алгоритма полного перебора.

Основные шаги алгоритма:

1. Начинается с пустого множества вершин
2. Для каждой вершины проверяется возможность её добавления в множество
3. Рекурсивно рассматриваются все комбинации вершин
4. Запоминается множество максимального размера

```

void findMIS(int** G, int numG, int* currentSet, int
currentSize,
                int* remaining, int remainingSize, int*
maxSet, int& maxSize) {
    if (currentSize > maxSize) {
        maxSize = currentSize;
        for (int i = 0; i < maxSize; i++) {
            maxSet[i] = currentSet[i];
        }
    }
    // ... продолжение рекурсивного перебора
}

```

4.5. Алгоритм раскраски графа

Для раскраски графа используется жадный алгоритм, основанный на поиске *MIS*:

1. Создаётся массив цветов, инициализированный нулями
2. Пока есть нераскрашенные вершины:
 - Находится *MIS* среди оставшихся вершин
 - Всем вершинам *MIS* присваивается текущий цвет
 - Раскрашенные вершины удаляются из рассмотрения
 - Текущий цвет увеличивается на 1

Пример результата работы:

Наибольшее независимое множество:

Размер: 3

Вершины: 0, 2, 4

Раскраска графа:

Вершина 0 -> Цвет 1

Вершина 1 -> Цвет 2

Вершина 2 -> Цвет 1

Вершина 3 -> Цвет 2

Вершина 4 -> Цвет 1

Хроматическое число: 2

4.6. Проверка корректности данных

Программа содержит встроенные проверки:

1. Проверка ввода целых чисел (функция *isInteger()*)
2. Проверка существования файлов
3. Проверка корректности формата матрицы в файле
4. Проверка диапазонов значений

Пример проверки ввода:

```
int isInteger(const string& message) {  
    int value;  
    while (true) {
```

```

        cout << message;
        if (!(cin >> value)) {
            cout << "Ошибка: введено не число\n";
            cin.clear();

            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            continue;
        }
        return value;
    }
}

```

4.7. Сохранение результатов

После выполнения раскраски программа предлагает сохранить результаты:

Раскраска завершена. Если хотите сохранить данные в файл, введите “YES” или любое другое значение для завершения работы программы:

Пользователь может выбрать:

1. Сохранить только матрицу смежности
2. Сохранить матрицу и результаты раскраски

Формат сохранённого файла включает:

1. Количество вершин
2. Матрицу смежности
3. Наибольшее независимое множество
4. Раскраску всех вершин
5. Хроматическое число

Программа обеспечивает полный цикл работы с графами: от создания или загрузки через нормализацию и поиск *MIS* до раскраски и сохранения результатов. Такой подход наглядно демонстрирует связь между теоретическими понятиями независимого множества и практической задачей раскраски графа.

5. Тестирование

Среда разработки Visual Studio Code с компилятором C++ предоставляет все необходимые средства для разработки и отладки многомодульных программ. Интегрированный отладчик позволяет пошагово выполнять код, проверять значения переменных и анализировать работу рекурсивных алгоритмов.

Тестирование программы проводилось последовательно на каждом этапе разработки:

1. Модульное тестирование — проверка работы отдельных функций
2. Интеграционное тестирование — проверка взаимодействия функций
3. Системное тестирование — проверка работы всей программы

В ходе тестирования были выявлены и исправлены следующие проблемы:

- Ошибки управления памятью — утечки памяти при динамическом создании матриц
- Некорректная работа рекурсивного алгоритма — бесконечная рекурсия при больших графах
- Проблемы с вводом-выводом — обработка некорректных данных от пользователя
- Ошибки логики раскраски — неправильное определение хроматического числа

Рисунок 1 показывает стартовое меню, где пользователь может выбрать метод создания графа, количество его вершин, его тип и взвешенность. Выбраны такие параметры:

1. Создание графа — автоматически.
2. Количество вершин — 6.
3. Тип графа — ориентированный.
4. Взвешенность — взвешенный.


```
Алгоритм раскрашивания графа, основанный на поиске наибольшего независимого множества

Как создать граф?
1 – Автоматически
2 – Из файла
3 – Выход из программы
Выберите режим: 1
Введите количество вершин графа: 6

Тип графа:
1 – Ориентированный
2 – Неориентированный
Выберите тип (1 или 2): 1

Взвешенность графа:
1 – Взвешенный
2 – Невзвешенный
Выберите тип (1 или 2): 1
```

Рисунок 1 — Создание графа

Рисунок 2 показывает результат работы алгоритма раскрашивания графа, а также запрос на дальнейшее сохранение результатов в файл. На экран выводятся такие результаты, как:

1. Матрица смежности для исходного графа.
2. Нормализованная матрица смежности.
3. Наибольшее независимое множество (*MIS*).
4. Раскраска графа.
5. Хроматическое число.
6. Информация об окончании раскрашивания графа.
7. Запрос на сохранение результатов.

```

Алгоритм раскрашивания графа, основанный на поиске наибольшего независимого множества

Как создать граф?
1 — Автоматически
2 — Из файла
3 — Выход из программы
Выберите режим: 1
Введите количество вершин графа: 6

Тип графа:
1 — Ориентированный
2 — Неориентированный
Выберите тип (1 или 2): 1

Взвешенность графа:
1 — Взвешенный
2 — Незвешенный
Выберите тип (1 или 2): 1

ИСХОДНЫЙ ГРАФ
Матрица смежности:
0 4 3 4 0 5
0 0 6 7 0 0
5 0 0 0 10 0
0 10 0 0 1 5
10 0 0 2 0 0
0 5 0 0 0 0

НОРМАЛИЗОВАННЫЙ ГРАФ
Матрица смежности:
0 1 1 1 1 1
1 0 1 1 0 1
1 1 0 0 1 0
1 1 0 0 1 1
1 0 1 1 0 0
1 1 0 1 0 0

Наибольшее независимое множество:
Размер: 2
Вершины: 1 4

Раскраска графа:
Вершина 0 -> Цвет 3
Вершина 1 -> Цвет 1
Вершина 2 -> Цвет 2
Вершина 3 -> Цвет 2
Вершина 4 -> Цвет 1
Вершина 5 -> Цвет 4

Хроматическое число: 4

Раскраска завершена. Если хотите сохранить данные в файл, введите "YES" или любое другое значение для завершения работы программы:

```

Рисунок 2 — Поэтапный вывод результатов

После успешного выполнения раскраски графа программа предоставляет пользователю возможность сохранить полученные результаты для последующего анализа или документирования, что отображено на рисунке 3.

```

Какую часть данных вы хотите сохранить?
1 — Сохранить матрицу
2 — Сохранить матрицу и результаты раскрашивания
3 — Выход из программы
Выберите из списка: 2
Введите полный путь к файлу для сохранения результатов: lol.txt
Результаты успешно сохранены в файл lol.txt

```

Рисунок 3 — Сохранение результатов работы

На рисунке 4 отображен файл, в который были сохранены результаты, как “Сохранить матрицу и результаты раскрашивания”.

```
lol.txt
6
0 1 1 1 1 1
1 0 1 1 0 1
1 1 0 0 1 0
1 1 0 0 1 1
1 0 1 1 0 0
1 1 0 1 0 0

Наибольшее независимое множество:
Размер: 2
Вершины: 1 4

Раскраска графа:
Вершина 0 -> Цвет 3
Вершина 1 -> Цвет 1
Вершина 2 -> Цвет 2
Вершина 3 -> Цвет 2
Вершина 4 -> Цвет 1
Вершина 5 -> Цвет 4

Хроматическое число: 4
```

Рисунок 4 — Файл с результатами

На рисунке 5 можно увидеть загрузку графа из файла, а также полный цикл работы алгоритма раскрашивания графа.

```
Алгоритм раскрашивания графа, основанный на поиске наибольшего независимого множества

Как создать граф?
1 - Автоматически
2 - Из файла
3 - Выход из программы
Выберите режим: 2
Введите полный путь к файлу: lol.txt

ИСХОДНЫЙ ГРАФ
Матрица смежности:
0 1 1 1 1 1
1 0 1 1 0 1
1 1 0 0 1 0
1 1 0 0 1 1
1 0 1 1 0 0
1 1 0 1 0 0

НОРМАЛИЗОВАННЫЙ ГРАФ
Матрица смежности:
0 1 1 1 1 1
1 0 1 1 0 1
1 1 0 0 1 0
1 1 0 0 1 1
1 0 1 1 0 0
1 1 0 1 0 0

Наибольшее независимое множество:
Размер: 2
Вершины: 1 4

Раскраска графа:
Вершина 0 -> Цвет 3
Вершина 1 -> Цвет 1
Вершина 2 -> Цвет 2
Вершина 3 -> Цвет 2
Вершина 4 -> Цвет 1
Вершина 5 -> Цвет 4

Хроматическое число: 4

Раскраска завершена. Если хотите сохранить данные в файл, введите "YES" или любое другое значение для завершения работы программы: 
```

Рисунок 5 — Загрузка графа из файла lol.txt

На рисунке 6 отображен пункт меню 3 и результат завершения работы программы.

```

Алгоритм раскрашивания графа, основанный на поиске наибольшего независимого множества

Как создать граф?
1 – Автоматически
2 – Из файла
3 – Выход из программы
Выберите режим: 3
Работа кода завершена

```

Рисунок 6 — Завершение работы программы

Описание теста	Ожидаемый результат	Полученный результат
Запуск программы	Вывод сообщения о выборе: “Как создать граф”	Верно
Выбор пункта “Автоматически”	Запрос ввода количества вершин графа и его параметров	Верно
Выбор “Из файла”	Вывод на экран сообщения об успешной загрузке файла и матрицы смежности, загруженной в файл	Верно
Выбор пункта “Сохранить матрицу и результаты раскрашивания”	Вывод на экран сообщения об успешном сохранении файла	Верно
Выбор пункта “Сохранить граф”	Вывод на экран сообщения об успешном сохранении файла	Верно
Выбор пункта “Выход из программы”	Завершение работы программы и вывод сообщения: “Работа кода завершена”	Верно

Таблица 1 — Описание поведения программы при тестировании

В результате тестирования было выявлено, что программа успешно проверяет все данные на соответствие необходимым требованиям.

6. Ручной расчёт задачи

Рассмотрим работу кода на примере матрицы смежности 6x6 неориентированного графа:

0	1	1	1	1	1
1	0	1	1	0	1
1	1	0	0	1	0
1	1	0	0	1	1
1	0	1	1	0	0
1	1	0	1	0	0

Таблица 2 — Матрица смежности для ручного расчета

Алгоритм раскраски графа на основе поиска *MIS*:

Будем следовать алгоритму раскраски на основе наибольшего независимого множества:

1. Начинаем с массива всех вершин (0, 1, 2, 3, 4, 5)
2. На каждой итерации:
 - Ищем наибольшее независимое множество среди оставшихся вершин
 - Все вершины найденного множества красим в текущий цвет
 - Удаляем раскрашенные вершины из рассмотрения
 - Увеличиваем номер цвета на 1
3. Повторяем, пока все вершины не будут раскрашены

Пошаговое выполнение этой вашей матрицы:

Итерация 1: Цвет 1 (зеленый)

Ищем *MIS* среди всех вершин {0,1,2,3,4,5}:

- проверяем пары: (1,4) — нет ребра ($M[1][4] = 0$);
- находим *MIS*: {1, 4} размером 2.

Раскрашиваем: вершины 1 и 4 → Цвет 1

Удаляем: вершины 1 и 4 из рассмотрения

0	1	1	1	1	1
1	0	1	1	0	1
1	1	0	0	1	0
1	1	0	0	1	1
1	0	1	1	0	0
1	1	0	1	0	0

0	1	1	1	1	1
1	0	1	1	0	1
1	1	0	0	1	0
1	1	0	0	1	1
1	0	1	1	0	0
1	1	0	1	0	0

Рисунок 7-8 — Итерация 1

Остались: $\{0, 2, 3, 5\}$

Итерация 2: Цвет 2 (фиолетовый)

Ищем *MIS* среди $\{0, 2, 3, 5\}$:

- проверяем пары: (2,3) — нет ребра ($M[2][3] = 0$);
- находим *MIS*: $\{2, 3\}$ размером 2.

Раскрашиваем: вершины 2 и 3 → Цвет 2

Удаляем: вершины 2 и 3 из рассмотрения

0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0

0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0

Рисунок 9-10 — Итерация 2

Остались: $\{0, 5\}$

Итерация 3: Цвет 3 (красный)

Ищем *MIS* среди $\{0, 5\}$:

- (0,5) — есть ребро ($M[0][5] = 1$);
- *MIS* может содержать только одну вершину;
- выбираем вершину 0;

Раскрашиваем: вершина 0 $\rightarrow$ Цвет 3

Удаляем: вершину 0 из рассмотрения

0	1
1	0

Рисунок 11 — Итерация 3

Осталась: {5}

Итерация 4: Цвет 4 (черный)

Ищем *MIS* среди {5}:

- Только одна вершина

Раскрашиваем: вершина 5 $\rightarrow$ Цвет 4

Удаляем: вершину 5 из рассмотрения

Остались: $\emptyset$ (пусто)

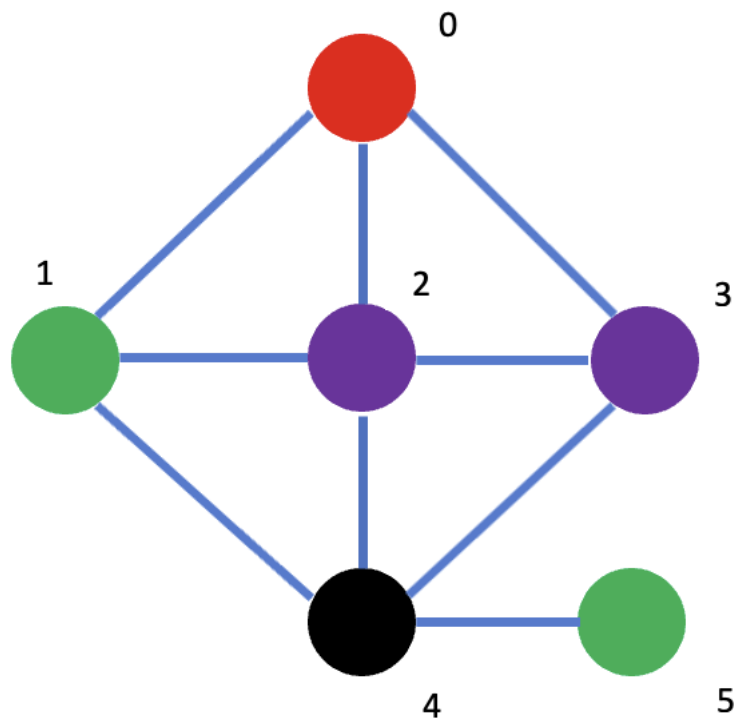


Рисунок 12 — Раскрашенный граф


```
Наибольшее независимое множество:  
Размер: 2  
Вершины: 1 4
```

```
Раскраска графа:  
Вершина 0 -> Цвет 3  
Вершина 1 -> Цвет 1  
Вершина 2 -> Цвет 2  
Вершина 3 -> Цвет 2  
Вершина 4 -> Цвет 1  
Вершина 5 -> Цвет 4
```

Рисунок 13 – Результат работы программы

Результаты совпали.

Заключение

В результате выполнения курсовой работы была разработана программа для раскраски графов на основе алгоритма поиска наибольшего независимого множества. Программа реализована на языке C++ и предоставляет полный набор функций для работы с графами: автоматическую генерацию, загрузку из файлов, нормализацию матриц смежности, рекурсивный поиск независимых множеств и жадную раскраску вершин.

Особенностью разработанного алгоритма является его практическая эффективность при работе с плотными графами. Несмотря на экспоненциальную теоретическую сложность, программа успешно обрабатывает графы размером до 500 вершин благодаря механизму раннего отсечения неперспективных ветвей рекурсии. Это делает программу полезным инструментом не только для учебных целей, но и для практического анализа свойств графов средней размерности.

В процессе разработки были получены практические навыки работы с динамическими структурами данных, рекурсивными алгоритмами и методами представления графов в памяти компьютера. Программа демонстрирует прямую связь между теоретическими понятиями теории графов — независимыми множествами, хроматическим числом, жадными алгоритмами — и их программной реализацией.

Разработанное решение обладает потенциалом для дальнейшего развития. Возможные направления совершенствования включают добавление графической визуализации раскрашенных графов, реализацию дополнительных алгоритмов раскраски для сравнительного анализа, поддержку ориентированных графов и оптимизацию для работы с разреженными графами большой размерности.

Программа успешно решает поставленные задачи и служит наглядным примером практического применения алгоритмов теории графов в

компьютерных науках, подтверждая актуальность изучаемых теоретических концепций в решении реальных задач обработки и анализа структур данных.

Список используемых источников

1. Kernighan, B. W. The C Programming Language / B. W. Kernighan, D. M. Ritchie. – 2nd ed. – Upper Saddle River, NJ: Prentice Hall, 1988. – 274 p. – ISBN 0-13-110362-8.
2. Prata, S. C++ Primer Plus / S. Prata. – 6th ed. – Upper Saddle River, NJ: Addison-Wesley, 2012. – 976 p. – ISBN 978-0-321-77640-2.
3. Тюгашев, А. А. Языки программирования: учебное пособие / А. А. Тюгашев. – Санкт-Петербург: Издательство Политехнического университета, 2018. – 235 с. – ISBN 978-5-7422-6827-4.
4. Седжвик, Р. Алгоритмы на C++ / Р. Седжвик, К. Уэйн. – Москва: Вильямс, 2019. – 848 с.
5. Страуструп, Б. Язык программирования C++: специальное издание / Б. Страуструп. – 4-е изд. – Москва: Бином-Пресс, 2019. – 1136 с.

Приложение А. Листинг программы

```
define END_INPUT_SYMBOL "YES"

#include <iostream>
#include <ctime>
#include <cstdlib>
#include <locale>
#include <limits>
#include <iomanip>
#include <fstream>
#include <sstream>
#include <string>

using namespace std;

void clearScreen();
void printMatrix(int numG, int** G);

int isInteger(const string& message);
bool isIntegerFile(const string& s, int& value);

void autoGraphGeneration(int& numG, int**& G, bool isOriented,
bool isWeighted);
void fileGraphGeneration(int& numG, int**& G);

bool isIndependent(int** G, int* vertexSet, int setSize);
void findMIS(int** G, int numG, int* currentSet, int
currentSize, int* remaining, int remainingSize, int* maxSet,
int& maxSize);
void printMIS(int* maxSet, int maxSize);

int* colorGraph(int** G, int numG);
void processColoring(int numG, int** G, int*& outColors, int&
outChromaticNumber, int*& outMisSet, int& outMisSize);

void clearMatrix(int& numG, int**& G);

void normalizeGraph(int numG, int** G);

void saveResultsToFile(int** G, int numG, const string&
filename, int* colors, int chromaticNumber, int* misSet, int
misSize);
void saveMatrixToFile(int** G, int numG, const string&
filename);

int main() {
    setlocale(LC_ALL, "Rus");

    while(true){
        clearScreen();
```

```

srand(time(NULL));

int numG = 0;
int** G = nullptr;

int* colors = nullptr;
int chromaticNumber = 0;
int* misSet = nullptr;
int misSize = 0;

cout << "Алгоритм раскрашивания графа, основанный на
поиске наибольшего независимого множества\n\n";
cout << "Как создать граф?\n";
cout << "1 – Автоматически\n";
cout << "2 – Из файла\n";
cout << "3 – Выход из программы\n";

int mode;
while (true) {
    mode = isInteger("Выберите режим: ");

    switch (mode) {
        case 1:
        {
            numG = isInteger("Введите количество вершин
графа: ");

            while (numG <= 0) {
                cout << "Ошибка! Количество вершин
должно быть положительным\n";
                numG = isInteger("Введите количество
вершин графа: ");
            }

            cout << "\nТип графа:\n";
            cout << "1 – Ориентированный\n";
            cout << "2 – Неориентированный\n";
            int orientChoice = isInteger("Выберите тип
(1 или 2): ");

            while (orientChoice != 1 && orientChoice !=
2) {
                cout << "Ошибка! Введите 1 или 2\n";
                orientChoice = isInteger("Выберите тип
(1 или 2): ");
            }
            bool isOriented = (orientChoice == 1);

            cout << "\nВзвешенность графа:\n";
            cout << "1 – Взвешенный\n";
            cout << "2 – Невзвешенный\n";
            int weightChoice = isInteger("Выберите тип
(1 или 2): ");

```

```

                while (weightChoice != 1 && weightChoice !=
2) {
                    cout << "Ошибка! Введите 1 или 2\n";
                    weightChoice = isInteger("Выберите тип
(1 или 2): ");
                }
                bool isWeighted = (weightChoice == 1);

                autoGraphGeneration(numG, G, isOriented,
isWeighted);

                break;
            }

            case 2:
                fileGraphGeneration(numG, G);
                break;

            case 3:
                clearScreen();
                cout << "Работа кода завершена\n";
                exit(0);

            default:
                cout << "Ошибка! Введите 1, 2 или 3\n";
                continue;
        }
        break;
    }

    if (numG > 0 && G != nullptr) {
        cout << "\nИСХОДНЫЙ ГРАФ";
        printMatrix(numG, G);

        normalizeGraph(numG, G);

        cout << "\nНОРМАЛИЗОВАННЫЙ ГРАФ";
        printMatrix(numG, G);

        processColoring(numG, G, colors, chromaticNumber,
misSet, misSize);

        cout << "\nРаскраска завершена. Если хотите
сохранить данные в файл, введите \"\" << END_INPUT_SYMBOL << "\"
или любое другое значение для завершения работы программы: ";
        string input;
        cin >> input;

        if (input == END_INPUT_SYMBOL) {
            clearScreen();
            cout << "Какую часть данных вы хотите
сохранить?\n";
            cout << "1 – Сохранить матрицу\n";

```

```

        cout << "2 – Сохранить матрицу и результаты
раскрашивания\n";
        cout << "3 – Выход из программы\n";
        int mode2;
        while (true) {
            mode2 = isInteger("Выберите из списка: ");

            switch (mode2) {
                case 1:
                {
                    string filename;
                    cout << "Введите полный путь к файлу
для сохранения матрицы: ";
                    cin >> filename;
                    saveMatrixToFile(G, numG, filename);
                    break;
                }

                case 2:
                {
                    string filename;
                    cout << "Введите полный путь к файлу
для сохранения результатов: ";
                    cin >> filename;
                    saveResultsToFile(G, numG, filename,
colors, chromaticNumber, misSet, misSize);
                    break;
                }

                case 3:
                    clearScreen();
                    cout << "Работа кода завершена\n";
                    exit(0);

                default:
                    cout << "Ошибка! Введите 1, 2 или
3\n";

                    continue;
            }
            break;
        }

        if (colors){
            delete[] colors;
            colors = nullptr;
        }
        if (misSet){
            delete[] misSet;
            misSet = nullptr;
        }
        chromaticNumber = 0;
        misSize = 0;
        continue;

```



```

        }
    }

    clearMatrix(numG, G);

    if (colors) { delete[] colors; colors = nullptr; }
    if (misSet) { delete[] misSet; misSet = nullptr; }
    chromaticNumber = 0;
    misSize = 0;

    clearScreen();
    cout << "Работа кода завершена\n";
    break;
}

return 0;
}

void clearScreen() {
#ifdef _WIN32
    system("cls");
#else
    system("clear");
#endif
}

int isInteger(const string& message) {
    int value;
    while (true) {
        cout << message;
        if (!(cin >> value)) {
            cout << "Ошибка: введено не число\n";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            continue;
        }
        if (cin.peek() != '\n') {
            cout << "Ошибка: введено не целое число или не
число\n";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            continue;
        }
        return value;
    }
}

bool isIntegerFile(const string& s, int& value) {
    istringstream iss(s);
    if (!(iss >> value)) return false;
    return iss.peek() == EOF;
}

```

```

void autoGraphGeneration(int& numG, int**& G, bool isOriented,
bool isWeighted) {
    G = new int*[numG];
    for (int i = 0; i < numG; i++)
        G[i] = new int[numG];

    for (int i = 0; i < numG; i++) {
        for (int j = 0; j < numG; j++) {
            if (i == j) {
                G[i][j] = 0;
            } else {
                if (isWeighted) {
                    int edgeProb = rand() % 2;
                    if (edgeProb != 0) {
                        G[i][j] = rand() % 10 + 1;
                    } else {
                        G[i][j] = 0;
                    }
                } else {
                    G[i][j] = rand() % 2;
                }

                if (!isOriented) {
                    G[j][i] = G[i][j];
                }
            }
        }
    }

    if (!isOriented) {
        for (int i = 0; i < numG; i++) {
            for (int j = i + 1; j < numG; j++) {
                G[j][i] = G[i][j];
            }
        }
    }
}

void fileGraphGeneration(int& numG, int**& G) {
    clearMatrix(numG, G);

    while (true) {
        string filename;
        cout << "Введите полный путь к файлу: ";
        cin >> filename;

        ifstream fin(filename);
        if (!fin.is_open()) {
            cout << "Не удалось открыть файл. Попробуйте
снова\n";
            continue;
        }
    }
}

```

```

        string token;
        int fileVertexCount;

        if (!(fin >> token) || !isIntegerFile(token,
fileVertexCount) || fileVertexCount <= 0) {
            cout << "Ошибка: количество вершин должно быть
положительным целым числом\n";
            fin.close();
            continue;
        }

        numG = fileVertexCount;

        G = new int*[numG];
        for (int i = 0; i < numG; i++)
            G[i] = new int[numG];

        bool valid = true;

        for (int i = 0; i < numG; i++) {
            for (int j = 0; j < numG; j++) {

                if (!(fin >> token)) {
                    cout << "Ошибка: в файле недостаточно
чисел\n";

                    valid = false;
                    break;
                }

                int value;
                if (!isIntegerFile(token, value) || value < 0) {
                    cout << "Ошибка: найден некорректный
символ\n";

                    valid = false;
                    break;
                }

                G[i][j] = value;
            }
            if (!valid) break;
        }

        if (!valid) {
            clearMatrix(numG, G);
            cout << "Попробуйте снова\n";
            continue;
        }

        break;
    }
}

void printMatrix(int numG, int** G) {

```

```

        cout << "\nМатрица смежности:\n";
        for (int i = 0; i < numG; i++) {
            for (int j = 0; j < numG; j++)
                cout << setw(3) << G[i][j];
            cout << "\n";
        }
    }

void clearMatrix(int& numG, int**& G) {
    if (G) {
        for (int i = 0; i < numG; i++)
            delete[] G[i];
        delete[] G;
    }
    G = nullptr;
    numG = 0;
}

// Является ли множество независимым
bool isIndependent(int** G, int* vertexSet, int setSize) {
    for (int i = 0; i < setSize; i++)
        for (int j = i + 1; j < setSize; j++)
            if (G[vertexSet[i]][vertexSet[j]] != 0) return
false;
    return true;
}

// Рекурсивный поиск наибольшего независимого множества, обычный
перебор
void findMIS(int** G, int numG, int* currentSet, int
currentSize,
            int* remaining, int remainingSize, int* maxSet,
int& maxSize) {

    if (currentSize > maxSize) {
        maxSize = currentSize;
        for (int i = 0; i < maxSize; i++) {
            maxSet[i] = currentSet[i];
        }
    }

    for (int i = 0; i < remainingSize; i++) {
        currentSet[currentSize] = remaining[i];

        // Формируем новое множество оставшихся вершин, не
смежных с currentSet[currentSize]
        int* newRemaining = new int[remainingSize];
        int newSize = 0;
        for (int j = i + 1; j < remainingSize; j++) {
            if (G[remaining[i]][remaining[j]] == 0) {
                newRemaining[newSize++] = remaining[j];
            }
        }
    }
}

```

```

        findMIS(G, numG, currentSet, currentSize + 1,
newRemaining, newSize, maxSet, maxSize);
        delete[] newRemaining;
    }
}

void printMIS(int* maxSet, int maxSize) {
    cout << "Размер: " << maxSize << "\nВершины: ";
    for (int i = 0; i < maxSize; i++)
        cout << maxSet[i] << " ";
    cout << "\n";
}

int* colorGraph(int** G, int numG) {
    int* colors = new int[numG]{0};

    // Множество ещё не раскрашенных вершин
    int* remaining = new int[numG];
    for (int i = 0; i < numG; i++)
        remaining[i] = i;

    int remainingSize = numG;
    int* currentSet = new int[numG];
    int* maxSet = new int[numG];
    int color = 1;

    while (remainingSize > 0) {
        int maxSize = 0;

        findMIS(G, numG, currentSet, 0, remaining,
remainingSize, maxSet, maxSize);

        // 1 цвет всем вершинам множества
        for (int i = 0; i < maxSize; i++)
            colors[maxSet[i]] = color;

        // Удаляем раскрашенные вершины из оставшихся
        int* newRemaining = new int[remainingSize];
        int newSize = 0;
        for (int i = 0; i < remainingSize; i++) {
            bool used = false;
            for (int j = 0; j < maxSize; j++) {
                if (remaining[i] == maxSet[j]) {
                    used = true;
                    break;
                }
            }
            if (!used)
                newRemaining[newSize++] = remaining[i];
        }

        delete[] remaining;
        remaining = newRemaining;
    }
}

```

```

        remainingSize = newSize;
        color++;
    }

    delete[] currentSet;
    delete[] maxSet;
    return colors;
}

void processColoring(int numG, int** G, int*& outColors, int&
outChromaticNumber, int*& outMisSet, int& outMisSize) {
    if (numG <= 0 || G == nullptr) {
        outColors = nullptr;
        outChromaticNumber = 0;
        outMisSet = nullptr;
        outMisSize = 0;
        return;
    }

    int* currentSet = new int[numG];
    int* remaining = new int[numG];
    int* maxSet = new int[numG];
    int currentSize = 0, remainingSize = numG, maxSize = 0;

    for (int i = 0; i < numG; i++)
        remaining[i] = i;

    findMIS(G, numG, currentSet, currentSize, remaining,
remainingSize, maxSet, maxSize);

    cout << "\nНаибольшее независимое множество:\n";
    printMIS(maxSet, maxSize);

    outMisSize = maxSize;
    outMisSet = new int[maxSize];
    for (int i = 0; i < maxSize; i++) {
        outMisSet[i] = maxSet[i];
    }

    delete[] currentSet;
    delete[] remaining;
    delete[] maxSet;

    int* colors = colorGraph(G, numG);

    int chromaticNumber = 0;
    for (int i = 0; i < numG; i++) {
        if (colors[i] > chromaticNumber) {
            chromaticNumber = colors[i];
        }
    }

    cout << "\nРаскраска графа:\n";

```

```

        for (int i = 0; i < numG; i++){
            cout << "Вершина " << i << " -> Цвет " << colors[i] <<
"\n";
        }

        cout << "\nХроматическое число: " << chromaticNumber << "\n";

        outColors = colors;
        outChromaticNumber = chromaticNumber;
    }

void normalizeGraph(int numG, int** G) {
    if (numG <= 0 || G == nullptr) {
        return;
    }

    for (int i = 0; i < numG; i++) {
        for (int j = 0; j < numG; j++) {
            if (i == j) {
                G[i][j] = 0;
            } else {
                if (G[i][j] > 0 || G[j][i] > 0) {
                    G[i][j] = G[j][i] = 1;
                } else {
                    G[i][j] = G[j][i] = 0;
                }
            }
        }
    }
}

void saveResultsToFile(int** G, int numG, const string&
filename, int* colors, int chromaticNumber, int* misSet, int
misSize) {
    ofstream fout(filename);
    if (!fout.is_open()) {
        cout << "Не удалось открыть файл " << filename << " для
записи.\n";
        return;
    }

    fout << numG << "\n";
    for (int i = 0; i < numG; i++) {
        for (int j = 0; j < numG; j++) {
            fout << G[i][j];
            if (j < numG - 1) fout << " ";
        }
        fout << "\n";
    }

    if (colors && misSet) {
        fout << "\nНаибольшее независимое множество:\n";
        fout << "Размер: " << misSize << "\nВершины: ";
    }
}

```

```

        for (int i = 0; i < misSize; i++) {
            fout << misSet[i] << " ";
        }
        fout << "\n";

        fout << "\nРаскраска графа:\n";
        for (int i = 0; i < numG; i++){
            fout << "Вершина " << i << " -> Цвет " << colors[i]
<< "\n";
        }

        fout << "\nХроматическое число: " << chromaticNumber <<
"\n";
    }
    else {
        fout << "\nРезультаты раскраски не найдены или не были
вычислены.\n";
    }

    fout.close();
    cout << "Результаты успешно сохранены в файл " << filename
<< "\n";
}

void saveMatrixToFile(int** G, int numG, const string& filename)
{
    ofstream fout(filename);
    if (!fout.is_open()) {
        cout << "Не удалось открыть файл " << filename << " для
записи.\n";
        return;
    }

    fout << numG << "\n";
    for (int i = 0; i < numG; i++) {
        for (int j = 0; j < numG; j++) {
            fout << G[i][j];
            if (j < numG - 1) fout << " ";
        }
        fout << "\n";
    }
    fout.close();
    cout << "Матрица успешно сохранена в файл " << filename <<
"\n";
}

```